

PRACTICAL RECORD

Node.js Fundamentals

Name: S Svara Haasini

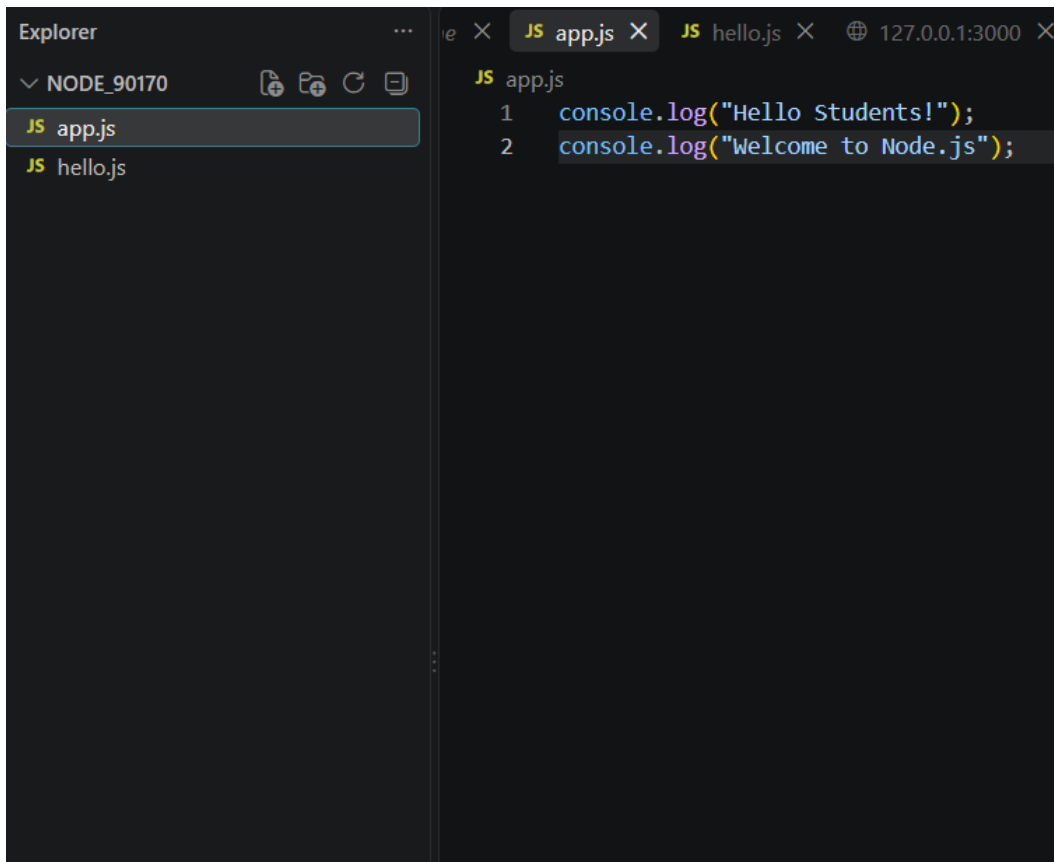
Roll Number: 2520090170

Aim:

To understand and execute basic Node.js programs demonstrating console output, HTTP servers, npm initialization, interactive console expressions, and JavaScript data types.

1. First Node.js Program

A basic Node.js program was executed using the node command to display messages in the terminal.

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a file named 'app.js' selected under the 'NODE_90170' workspace. The main editor area displays the code for 'app.js' with two lines: '1 console.log("Hello Students!");' and '2 console.log("Welcome to Node.js");'. The code is syntax-highlighted, with 'console.log' in blue, strings in red, and the file name 'app.js' in the tab and editor header in yellow. The background is dark, and the text is light-colored.

```
JS app.js
1 console.log("Hello Students!");
2 console.log("Welcome to Node.js");
```

Figure 1: First Node.js Program - app.js Code

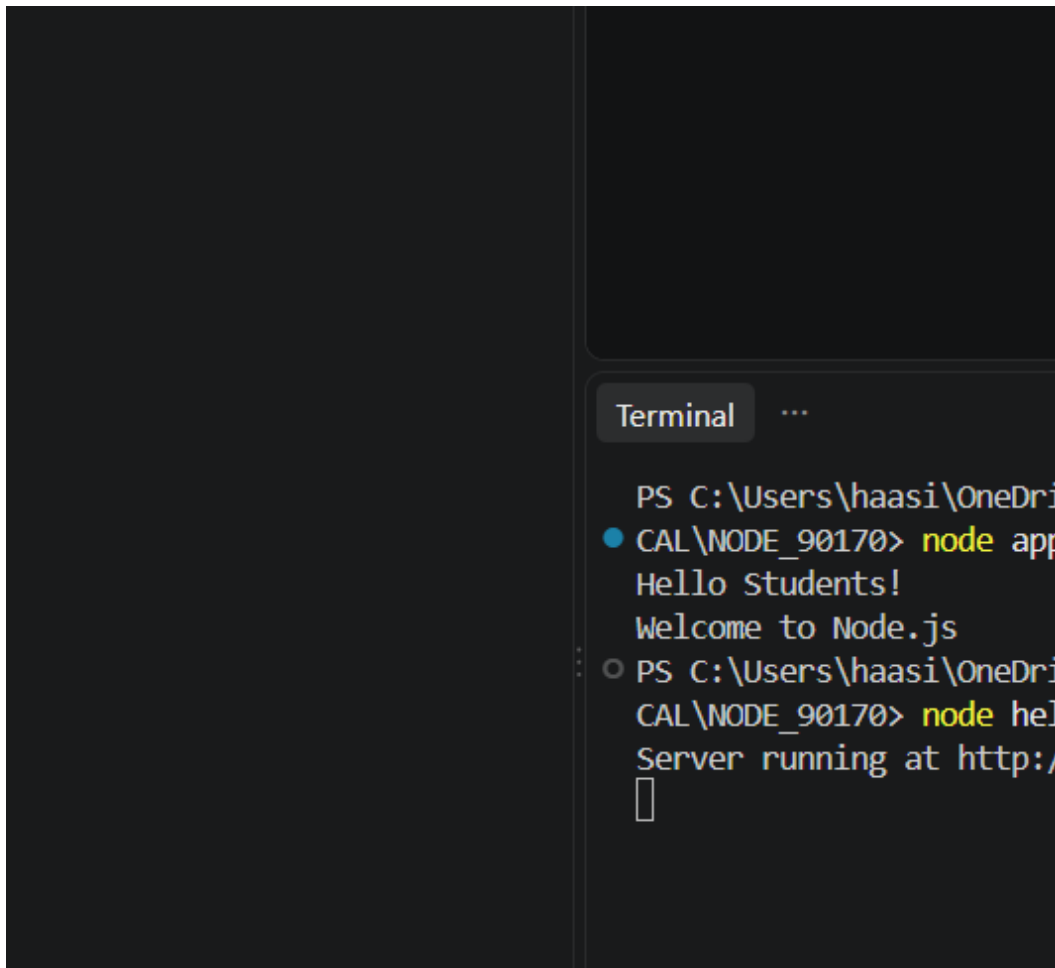
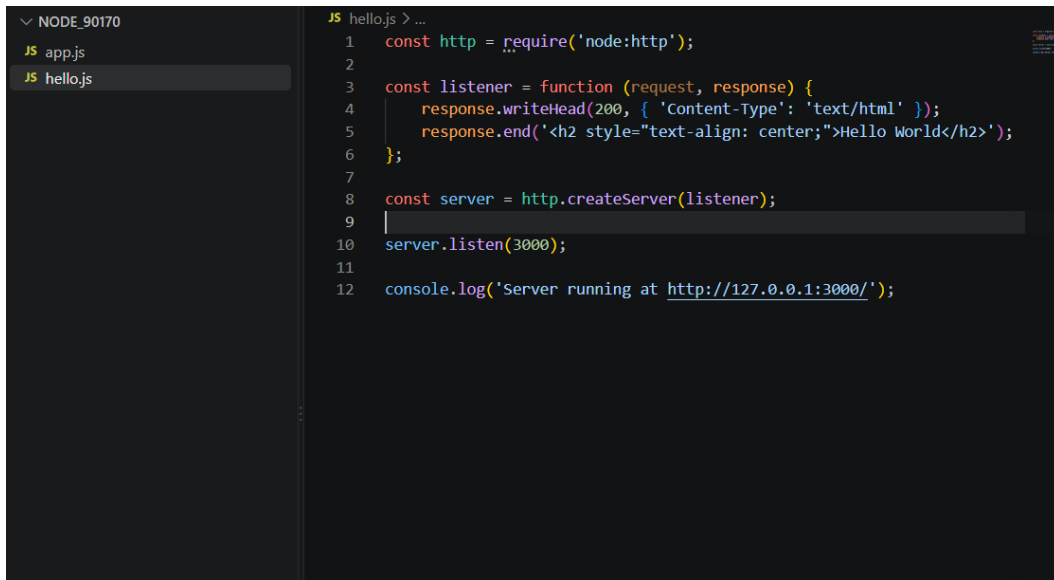
A screenshot of a terminal window with a dark background. The terminal shows a PowerShell prompt 'PS C:\Users\haasi\OneDri' followed by a command 'CAL\NODE_90170> node app'. The output of the command is 'Hello Students!' and 'Welcome to Node.js'. Below this, the prompt is shown again with the command 'CAL\NODE_90170> node hel', and the output is 'Server running at http:/' followed by a cursor. The terminal window has a tab labeled 'Terminal' and a menu icon '...'.

Figure 2: First Node.js Program - Terminal Output

2. Hello World HTTP Server

A basic HTTP server was created using Node.js HTTP module and accessed through a web browser on port 3000.



```
JS hello.js > ...
1  const http = require('node:http');
2
3  const listener = function (request, response) {
4      response.writeHead(200, { 'Content-Type': 'text/html' });
5      response.end('<h2 style="text-align: center;">Hello World</h2>');
6  };
7
8  const server = http.createServer(listener);
9
10 server.listen(3000);
11
12 console.log('Server running at http://127.0.0.1:3000/');
```

Figure 3: Hello World HTTP Server - hello.js Code

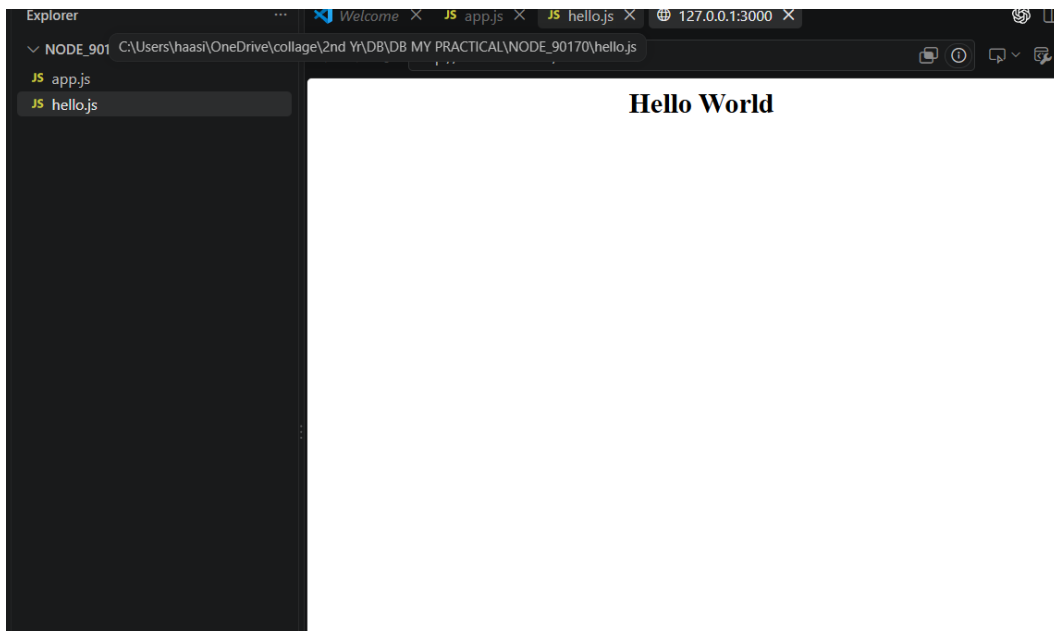
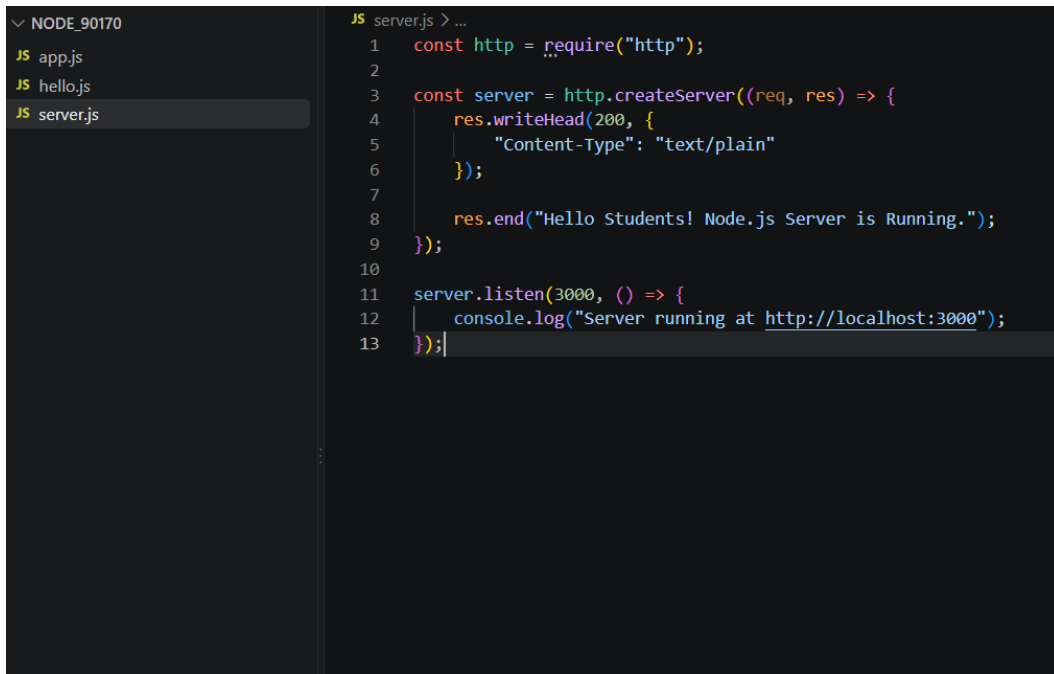


Figure 4: Hello World HTTP Server - Browser Output

3. Simple Node.js HTTP Server

A simple HTTP server was created to return a text response to the browser at localhost:3000.



```
JS server.js > ...
1  const http = require("http");
2
3  const server = http.createServer((req, res) => {
4      res.writeHead(200, {
5          "Content-Type": "text/plain"
6      });
7
8      res.end("Hello Students! Node.js Server is Running.");
9  });
10
11 server.listen(3000, () => {
12     console.log("Server running at http://localhost:3000");
13 });
```

Figure 5: Simple Node.js HTTP Server - server.js Code

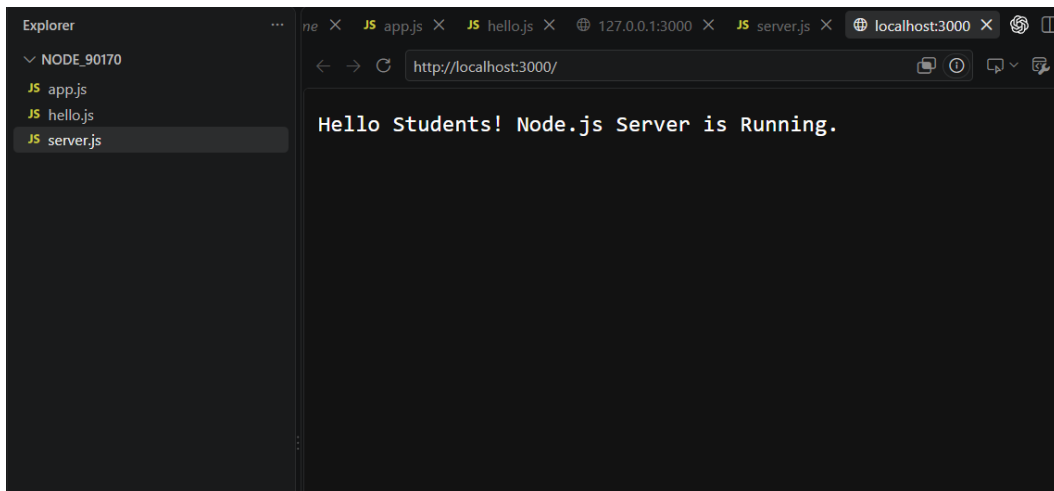
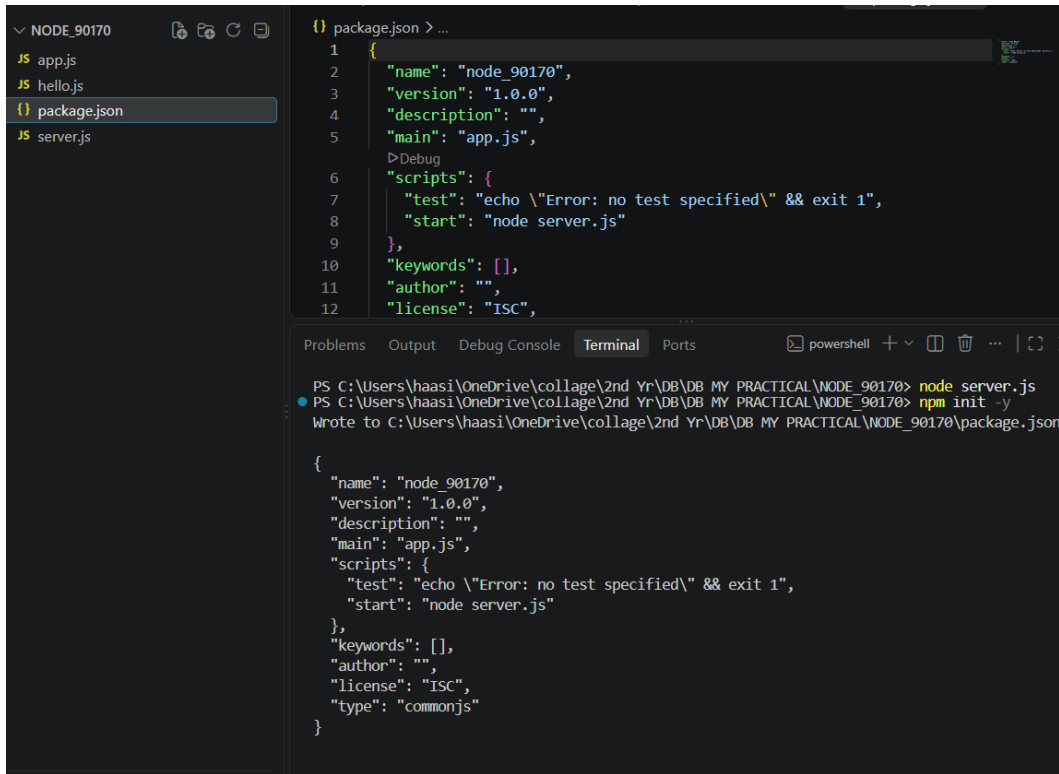


Figure 6: Simple Node.js HTTP Server - Browser Output

4. npm Initialization

npm was initialized in the project folder using `npm init -y` to create the package.json configuration file.



The image shows a Visual Studio Code editor with a file explorer on the left containing 'app.js', 'hello.js', 'package.json', and 'server.js'. The main editor displays the 'package.json' file with the following content:

```
1 {
2   "name": "node_90170",
3   "version": "1.0.0",
4   "description": "",
5   "main": "app.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1",
8     "start": "node server.js"
9   },
10  "keywords": [],
11  "author": "",
12  "license": "ISC",
13 }
```

The bottom panel shows the 'Terminal' tab with the following output:

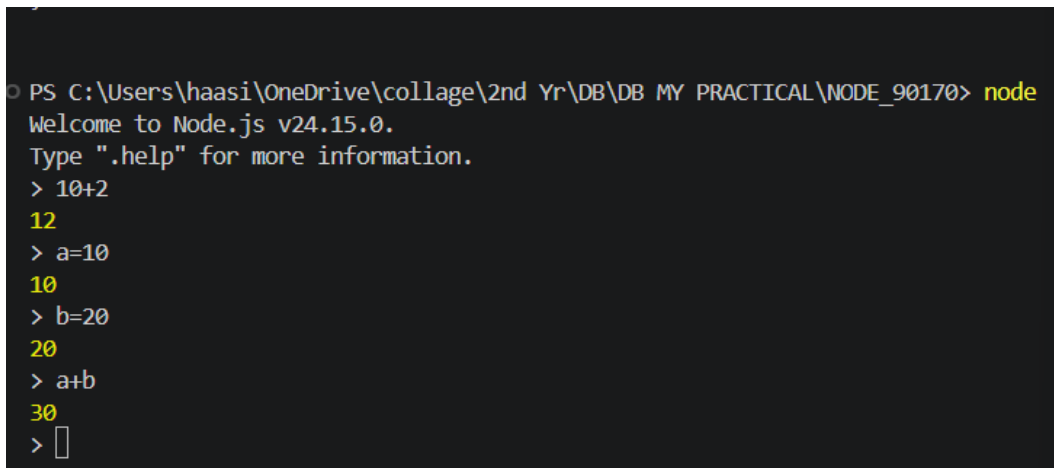
```
PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170> node server.js
PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170> npm init -y
Wrote to C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170\package.json

{
  "name": "node_90170",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Figure 7: npm Initialization - package.json and Terminal Output

5. Node.js Simple Expressions

Basic JavaScript expressions and variables were tested directly using the Node.js interactive console.



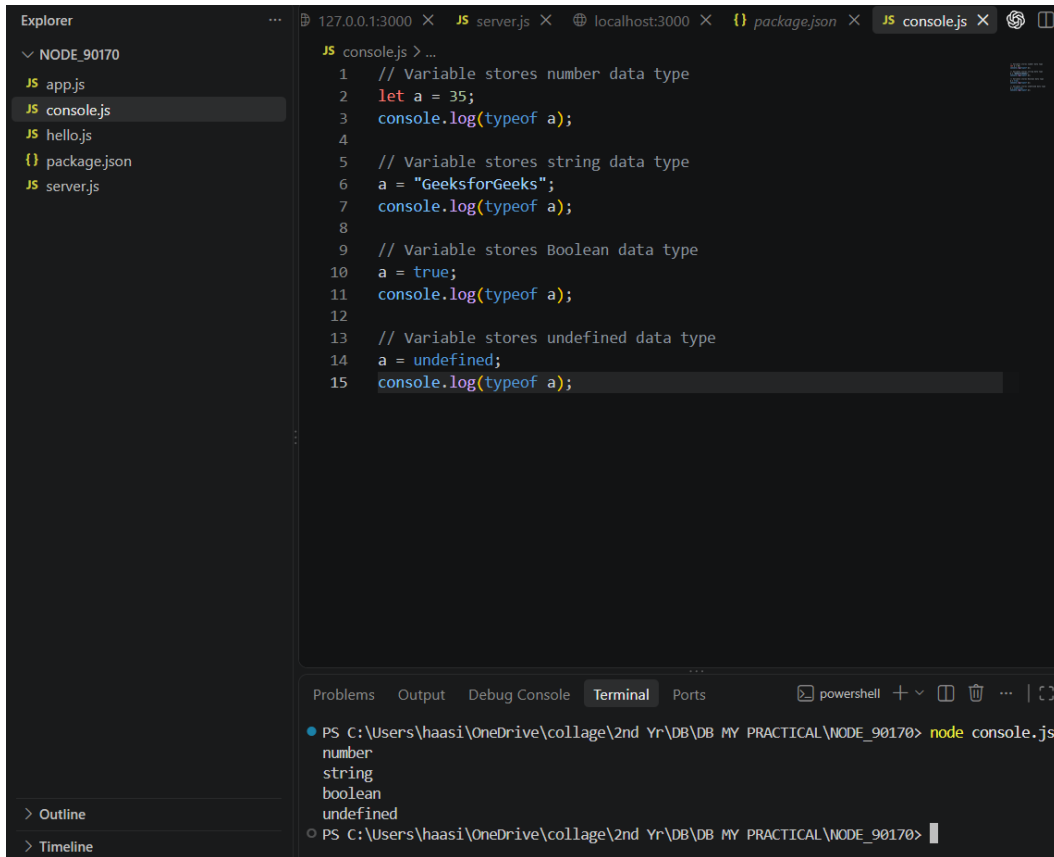
The image shows the Node.js interactive console with the following output:

```
PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170> node
Welcome to Node.js v24.15.0.
Type ".help" for more information.
> 10+2
12
> a=10
10
> b=20
20
> a+b
30
> 
```

Figure 8: Node.js Interactive Console - Expressions and Variables

6. Node.js Data Types

The `typeof` operator was used in Node.js to identify different JavaScript data types including number, string, boolean, and undefined.



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'NODE_90170' with files 'app.js', 'console.js', 'hello.js', 'package.json', and 'server.js'. The 'console.js' file is selected and its code is displayed in the editor. The code defines three variables: a number, a string, and a boolean, each followed by a `console.log(typeof a);` statement. It also defines an undefined variable. The terminal at the bottom shows the command `node console.js` being executed, and the output displays the data types: 'number', 'string', 'boolean', and 'undefined'.

```
1 // Variable stores number data type
2 let a = 35;
3 console.log(typeof a);
4
5 // Variable stores string data type
6 a = "GeeksforGeeks";
7 console.log(typeof a);
8
9 // Variable stores Boolean data type
10 a = true;
11 console.log(typeof a);
12
13 // Variable stores undefined data type
14 a = undefined;
15 console.log(typeof a);
```

PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170> node console.js
number
string
boolean
undefined
PS C:\Users\haasi\OneDrive\collage\2nd Yr\DB\DB MY PRACTICAL\NODE_90170>

Figure 9: Node.js Data Types - console.js Code and Terminal Output

The Node.js programs were executed successfully, demonstrating basic JavaScript execution, HTTP server creation, npm initialization, Node.js console expressions, and JavaScript data types.

7. Password Program

This Node.js program demonstrates password hashing using the `bcrypt` module for secure password storage.

The image shows a Visual Studio Code editor with a project named 'NODE_90170'. The file explorer on the left lists several files: 'node_modules', 'app.js', 'clientserver.js', 'console.js', 'hello.js', 'package-lock.json', 'package.json', 'password.js' (selected), 'rbac.js', and 'server.js'. The main editor displays the code for 'password.js', which uses the 'bcrypt' library to hash a password. The code is as follows:

```
1 const bcrypt = require("bcrypt");
2 const password = "Student123";
3 bcrypt.hash(password, 10, (err, hash) => {
4   if(err){
5     console.log(err);
6   } else{
7     console.log("Password Hash:");
8     console.log(hash);
9   }
10 });
```

At the bottom, the integrated terminal shows the execution of the program. It starts with the command 'npm install bcrypt', which successfully installs the package. Then, the command 'node password.js' is run, resulting in the output: 'Password Hash: \$2b\$10\$g5nLhui3eVPYDS5NA2cuBedySkNJTRxsZHuItBjIqKYxgMMk8/SEK'.

Figure 10: Node.js Password Program with bcrypt Hashing

8. Client-Server Communication

The Node.js client and server programs demonstrate HTTP-based communication between a server and client on port 3000.

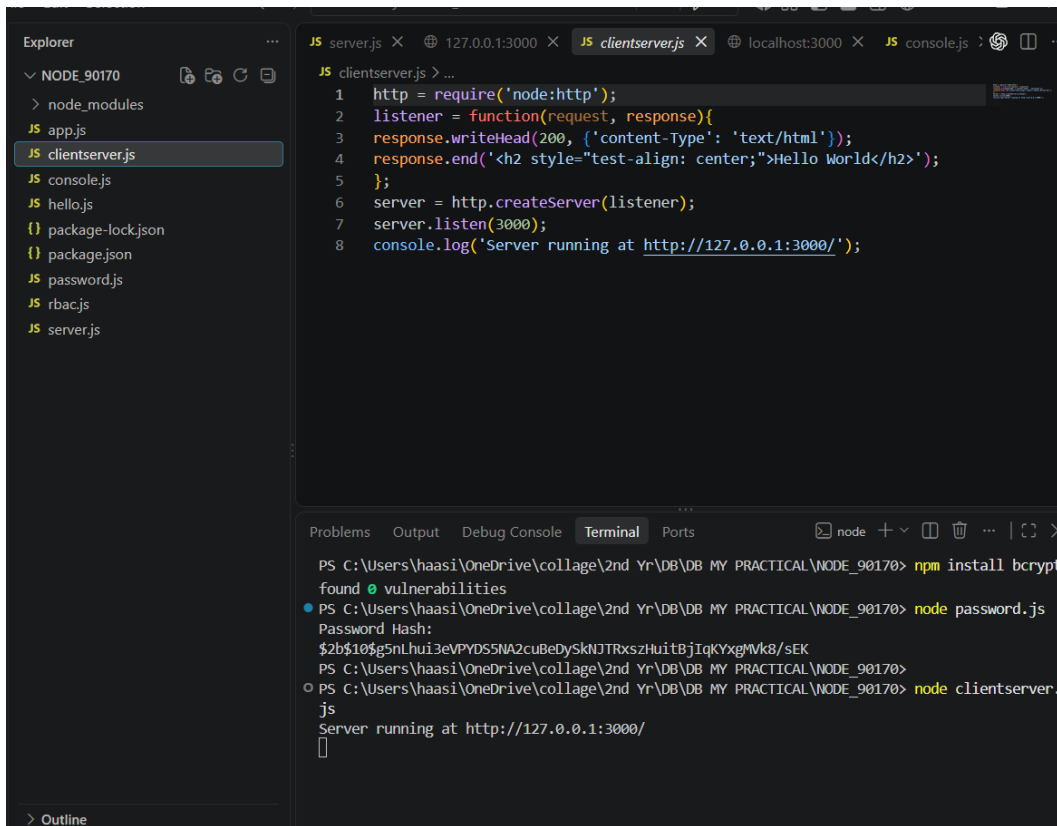


Figure 11: Node.js Client-Server Program Code and Terminal Output

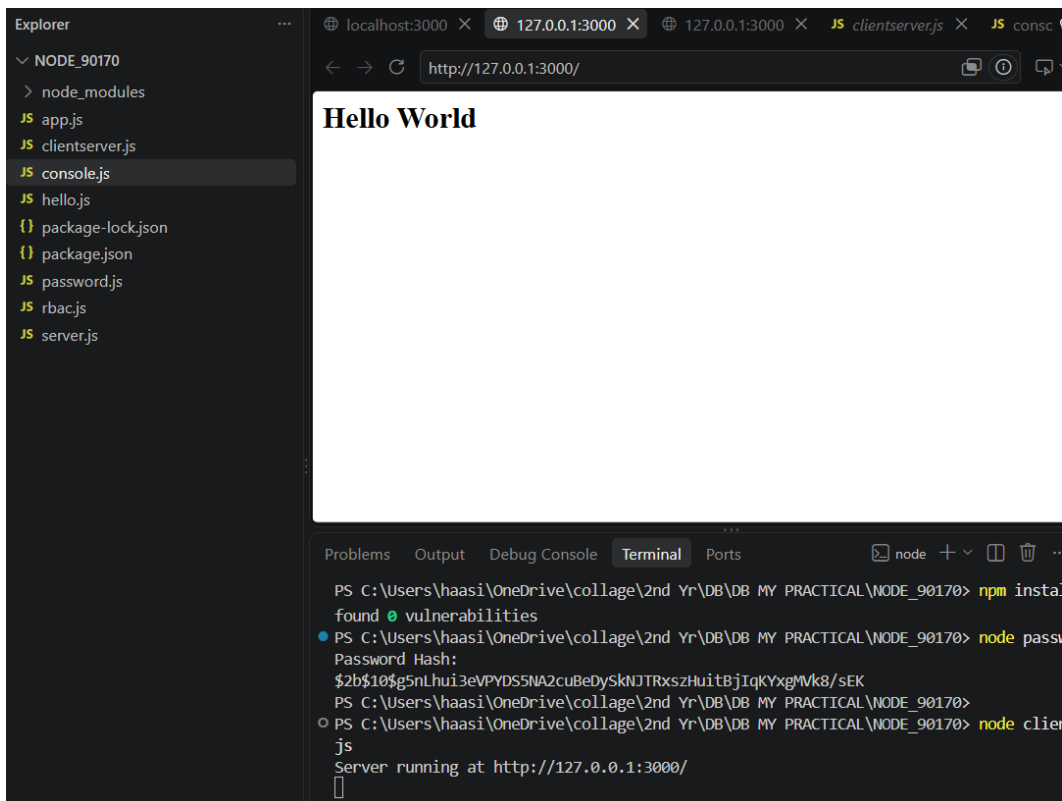


Figure 12: Client-Server Program Browser Output

Result:

The Node.js programs were executed successfully, demonstrating basic JavaScript execution, HTTP server creation, npm initialization, Node.js console expressions, JavaScript data types, password hashing with bcrypt, and client-server communication.